Smart Study Planner: An Algorithmic Approach to Productivity Optimization

Subject: Algorithmic Thinking

Institution: SRM University AP

Faculty: Dr.Prakash

Submission Date: 30-04-2026

Team Members:

- Chetan Saatvic Reddy Beeram – AP24110010879
- Uppugunduri S V Srichaithra – AP24110010945
- Thokala Srivignesh – AP24110010813
- Mallela Rushyanth – AP24110010888
- Thanuku Chandra Shekhar – AP24110010863

Abstract

Effective academic planning is a critical determinant of student success, yet traditional methods often lack the adaptability and optimization necessary to manage complex workloads efficiently. This report presents the design and implementation of the **Smart Study Planner**, a decision support system engineered to generate optimized study schedules through the application of algorithmic logic. Addressing the real-world challenge of balancing multiple academic commitments, the system employs a novel **Dynamic Greedy Heuristic model** coupled with a sophisticated priority function to allocate study time for diverse subjects. The approach prioritizes tasks based on difficulty, proximity to deadlines, and dynamically adjusts priorities to ensure fairness and prevent subject fatigue. The outcome is an interactive, data-driven study schedule visualized through a Streamlit-based user interface, providing students with a personalized and efficient pathway to academic achievement.

1. Introduction

In the contemporary educational landscape, students are frequently confronted with a multifaceted array of academic responsibilities, including coursework, assignments, examinations, and extracurricular activities. The effective management of these commitments is paramount for academic excellence and overall well-being. Traditional study planning methodologies, often reliant on manual scheduling or rudimentary digital calendars, frequently prove inadequate in optimizing time allocation, leading to suboptimal performance, increased stress, and potential burnout. The inherent complexity of balancing

varying subject difficulties, fluctuating deadlines, and individual learning capacities necessitates a more sophisticated approach.

This project addresses this exigency by leveraging the power of algorithmic thinking to develop a **Smart Study Planner**. Algorithms, as systematic procedures for solving computational problems, offer a robust framework for optimizing resource allocation, a principle directly applicable to study scheduling. By transforming the study planning dilemma into an optimization problem, we can harness algorithmic logic to generate schedules that are not only efficient but also adaptive and equitable. This report details the conceptualization, design, and implementation of such a system, emphasizing its algorithmic core and its utility as a decision support tool for students.

2. Problem Statement

The pervasive challenge among students is the inefficient allocation of study time across multiple subjects, often resulting in last-minute cramming, neglect of certain topics, or an imbalanced workload. This issue is exacerbated by the static nature of conventional planners, which fail to dynamically adjust to changes in task priorities, unforeseen events, or the inherent variability in learning processes. The absence of an intelligent system that can quantitatively assess task parameters (e.g., difficulty, deadline proximity) and generate an optimized, fair, and adaptive study regimen constitutes a significant impediment to academic productivity and student welfare. Therefore, the problem is to design and implement a system that can effectively mitigate these challenges by providing an algorithmically optimized and dynamically adjustable study schedule.

3. Objectives

The primary objectives of the Smart Study Planner project are:

- To design and implement a robust algorithmic framework capable of generating optimized study schedules.
- To develop a priority function that quantitatively assesses the urgency and importance of study tasks based on difficulty and deadlines.
- To incorporate a fairness constraint within the scheduling algorithm to prevent over-allocation to a single subject and mitigate study fatigue.
- To create an intuitive and interactive user interface for inputting study parameters and visualizing the generated schedules and analytics.
- To provide real-time analytical insights into study progress and workload distribution through charts and metrics.
- To enable flexible syllabus management, including interactive editing and data export functionalities.
- To deliver a professional, university-level academic report detailing the system's design, implementation, and evaluation.

4. Literature Survey

The landscape of study planning tools ranges from simple calendar applications to more sophisticated digital planners. Existing solutions like Google Calendar, Microsoft To Do, and various mobile applications offer basic scheduling functionalities, task management, and reminder systems. However, these tools primarily serve as digital repositories for tasks, requiring manual input and lacking inherent intelligence for optimization. They do not typically incorporate algorithmic logic to suggest optimal study times based on task attributes or student-specific learning patterns.

More advanced academic planners, such as those integrated into Learning Management Systems (LMS) like Canvas or Blackboard, provide assignment tracking and due date reminders but similarly fall short in offering dynamic, algorithmically optimized scheduling. While some commercial productivity applications claim to use AI for task prioritization, their underlying methodologies are often opaque and lack the explicit fairness constraints crucial for academic well-being. Limitations across these existing systems include:

- **Lack of Optimization:** Schedules are typically static and do not adapt to changing priorities or new inputs.
- **Absence of Fairness:** No inherent mechanism to prevent disproportionate allocation of time to a single subject, leading to potential burnout.
- **Limited Customization:** Generic scheduling rules that do not account for individual subject difficulty or learning pace.
- **Poor Visualization:** Often present schedules in a linear, non-analytical format, lacking insights into workload distribution or progress.
- **No Decision Support:** Act as mere organizers rather than intelligent systems guiding optimal study decisions.

This survey highlights a significant gap in the market for a truly intelligent, algorithmically driven study planner that prioritizes fairness and dynamic optimization, which the Smart Study Planner aims to address.

5. Proposed System: Smart Study Planner

The **Smart Study Planner** is envisioned as a sophisticated decision support system designed to revolutionize academic scheduling. It moves beyond conventional static planners by integrating advanced algorithmic logic to generate personalized, optimized, and fair study schedules. The system's core functionality revolves around intelligent allocation of study hours, taking into account various parameters to maximize learning efficiency and minimize academic stress.

Key Features:

- **Intelligent Scheduling:** Utilizes a **Dynamic Greedy Heuristic model** to create schedules based on subject difficulty, remaining days until deadlines, and allocated study hours.

- **Fairness Constraint:** Actively prevents subject fatigue by ensuring a balanced distribution of study time across all subjects, avoiding excessive focus on a single area.
- **Real-time Analytics:** Provides interactive charts and metrics (e.g., progress trackers, workload distribution) to offer insights into study patterns and efficiency.
- **Interactive Syllabus Editing:** Allows users to easily add, modify, or remove subjects and their associated parameters (difficulty, total hours, deadline).
- **CSV Export:** Facilitates data portability by enabling users to export their generated schedules and analytics in CSV format.
- **SaaS-style UI:** Features a modern, intuitive user interface built with Streamlit, incorporating a glassmorphism design for an aesthetically pleasing and functional experience.

6. Algorithmic Approach (CORE)

The heart of the Smart Study Planner lies in its **Dynamic Greedy Heuristic model**. This model is designed to make locally optimal choices at each stage with the hope of finding a global optimum. The
greedy approach is particularly suitable for scheduling problems where decisions need to be made sequentially based on current conditions.

Priority Function

To effectively prioritize study tasks, a comprehensive priority function (P) has been developed. This function quantifies the urgency and importance of a subject based on its perceived difficulty and the proximity of its deadline. The formula is defined as:

```
P = (Difficulty × 3) + (20 / DaysLeft)
```

Where:

- **Difficulty:** An integer value representing the perceived difficulty of the subject (e.g., 1 for easy, 5 for very hard). The multiplier of 3 emphasizes the impact of difficulty on overall priority.
- **DaysLeft:** The number of days remaining until the subject's deadline. The inverse relationship ensures that subjects with closer deadlines receive higher priority. The constant 20 scales the impact of days left.

Dynamic Priority

To ensure the schedule remains adaptive and responsive to ongoing study efforts, a dynamic adjustment is applied to the base priority. As hours are allocated to a subject, its priority is incrementally reduced, reflecting the progress made and allowing other subjects to gain prominence. This dynamic priority (P_dynamic) is calculated as:

```
P_dynamic = BasePriority − (AllocatedHours × 1.5)
```

Where:

- **BasePriority:** The initial priority calculated using the primary priority function.
- **AllocatedHours:** The total hours already allocated to the subject in the current scheduling iteration. The multiplier of 1.5 ensures a significant reduction in priority as more hours are assigned, promoting a balanced allocation.

Fairness Constraint and Decision-Making

A critical aspect of the Smart Study Planner is its **fairness constraint**, designed to prevent subject fatigue and ensure a balanced academic workload. This constraint operates by limiting the consecutive hours allocated to a single subject within a given scheduling block. For instance, after a subject has been allocated a certain number of hours, the algorithm temporarily reduces its priority or prevents further allocation until other subjects have received attention. This mechanism ensures that all subjects are progressed in a balanced manner, mitigating the risk of burnout and promoting holistic learning.

Decision-making within the greedy heuristic model involves iterating through all available subjects, calculating their dynamic priority, and selecting the subject with the highest priority for allocation of a study block (e.g., 1 hour). This process is repeated until all available study hours are allocated or all subjects have met their required study time. The fairness constraint is applied at each decision point, influencing the selection process to maintain balance.

Pseudocode

```
Function GenerateStudySchedule(Subjects, TotalStudyHours):

  Schedule = EmptyList
  AllocatedHoursPerSubject = Dictionary(Subject -> 0)
  LastAllocatedSubject = None

  While TotalStudyHours > 0:
    HighestPriority = -Infinity
    SubjectToAllocate = None

    For Each Subject in Subjects:
      BasePriority = (Subject.Difficulty * 3) + (20 / Subject.DaysLeft)
      DynamicPriority = BasePriority - (AllocatedHoursPerSubject[Subject] * 1.5)

      // Apply Fairness Constraint
      If Subject == LastAllocatedSubject and AllocatedHoursPerSubject[Subject] % MaxConsecutiveHours
== 0:
        Continue // Skip this subject for now to ensure fairness

      If DynamicPriority > HighestPriority:
```

```
    HighestPriority = DynamicPriority
    SubjectToAllocate = Subject

  If SubjectToAllocate is None:
    Break // No more subjects to allocate or all constraints met

  Add SubjectToAllocate to Schedule
  AllocatedHoursPerSubject[SubjectToAllocate] += 1
  LastAllocatedSubject = SubjectToAllocate
  TotalStudyHours -= 1


Return Schedule
```

Complexity

- **Time Complexity: O(n log n)** The primary operations involve iterating through 'n' subjects to calculate priorities and selecting the highest priority subject. If a priority queue or similar data structure is used for efficient selection, each selection operation can be O(log n). Since this is done for each hour allocated (up to 'n' hours in the worst case, considering each subject needs some allocation), the overall time complexity approaches O(n log n), where 'n' is the number of subjects or study blocks.
- **Space Complexity: O(n)** The space complexity is primarily determined by the storage required for subject information, the schedule list, and the dictionary tracking allocated hours per subject. All these data structures scale linearly with the number of subjects 'n'.

7. System Design

The Smart Study Planner is architected as a modular system, promoting maintainability, scalability, and clear separation of concerns. The design emphasizes a client-server model, where the Streamlit application acts as the client-side interface, interacting with a backend logic module responsible for algorithmic computations and data management.

Architecture & Modules

```
graph TD

  A[User] -->|Interacts with| B(Streamlit UI)
  B -->|Sends Input to| C{Planner Core Logic}
  C -->|Applies Algorithms| D[Dynamic Greedy Heuristic]
  D -->|Generates| E[Optimized Schedule]
  C -->|Manages Data| F[Pandas DataFrames]
  B -->|Visualizes| G[Plotly Express Charts]
  E -->|Displayed on| B
```

```
F -->|Displayed on| B
```

Modules:

1. **Streamlit UI (app.py):** This module serves as the primary interface for the user. It handles:
    - Input forms for subject details (name, difficulty, total hours, deadline).
    - Display of the generated study schedule.
    - Visualization of analytics (charts, metrics) using Plotly Express.
    - Interactive controls for editing subjects and exporting data.
    - Integration of CSS for a modern glassmorphism design.
2. **Planner Core Logic (planner.py):** This is the computational backbone of the system, encapsulating the algorithmic intelligence. It contains:
    - Implementation of the Subject class (OOP).
    - The PriorityFunction and DynamicPriority calculations.
    - The GenerateStudySchedule algorithm, including the fairness constraint.
    - Functions for managing subject data (add, edit, delete) and persisting it.
3. **Data Management (Pandas):** Utilized within planner.py to efficiently store, manipulate, and retrieve subject data and generated schedule details. Pandas DataFrames provide a robust and flexible structure for handling tabular data.
4. **Visualization (Plotly Express):** Integrated into app.py to render interactive and informative charts, providing users with a clear overview of their study progress, workload distribution, and subject priorities.

File Structure

The project adheres to a clear and organized file structure to facilitate development, deployment, and understanding:

```
./
```

```
├── app.py
├── planner.py
├── requirements.txt
├── README.md
```

```
└── screenshot.png
```

- app.py: Contains the Streamlit application code, handling the user interface and interactions.
- planner.py: Encapsulates the core algorithmic logic, subject management, and schedule generation.

- requirements.txt: Lists all Python dependencies required to run the project (e.g., streamlit, pandas, plotly).
- README.md: Provides a comprehensive overview of the project, setup instructions, and usage guidelines.
- screenshot.png: A visual representation of the application's dashboard or key output.

8. Implementation

The Smart Study Planner is implemented using a modern and efficient tech stack, primarily focusing on Python for its robust data handling capabilities and the Streamlit framework for rapid web application development. The object-oriented programming (OOP) paradigm is adopted to ensure modularity, reusability, and maintainability of the codebase.

Tech Stack

- **Python (OOP):** The primary programming language, used for all backend logic, algorithmic implementation, and data processing. OOP principles are applied to define classes for Subject and potentially Scheduler to manage their attributes and behaviors.
- **Streamlit + CSS:** Streamlit is employed for building the interactive web-based user interface. Its simplicity and rapid prototyping capabilities allow for quick development of data applications. Custom CSS is integrated to achieve the desired SaaS-style glassmorphism aesthetic, enhancing the user experience.
- **Pandas:** A powerful data manipulation and analysis library in Python. Pandas DataFrames are used to store and manage subject information, study hours, deadlines, and the generated schedule, providing efficient data handling.
- **Plotly Express:** A high-level Python library for creating interactive, publication-quality graphs. It is used to visualize the study schedule, workload distribution, and other key analytics within the Streamlit dashboard.

Workflow

The system operates through a clear and sequential workflow:

5 **Input:** The user interacts with the Streamlit UI (app.py) to input details for each subject, including its name, estimated difficulty, total required study hours, and the deadline. This data is then processed and stored, typically in a Pandas DataFrame.

6 **Processing:** The planner.py module takes the input subject data. It then applies the Dynamic Greedy Heuristic algorithm, utilizing the defined priority functions and fairness constraints, to generate an optimized study schedule. This involves iteratively allocating study blocks to subjects based on their dynamic priorities until all study hours are assigned or subject requirements are met.

7 **Visualization:** The generated schedule and associated analytics are then passed back to app.py. Plotly Express is used to render interactive charts and a clear tabular representation of the schedule on the Streamlit dashboard. This allows the user to

visually understand their study plan, monitor progress, and identify potential areas for adjustment.

9. Results & Output

The Smart Study Planner delivers a comprehensive and interactive output designed to empower students with actionable insights into their study regimen. The primary output is presented through a dynamic dashboard, accessible via the Streamlit application.

Dashboard

The main dashboard provides a holistic view of the study plan, featuring:

- **Generated Study Schedule:** A clear, day-by-day breakdown of allocated study hours per subject, presented in a tabular format. This schedule is the direct result of the algorithmic optimization.
- **Real-time Analytics:** Interactive charts and graphs (powered by Plotly Express) illustrating:
  - **Workload Distribution:** A pie chart or bar chart showing the proportion of study hours allocated to each subject.
  - **Progress Tracking:** A line graph or bar chart depicting the cumulative hours allocated versus the total required hours for each subject.
  - **Priority Evolution:** A visualization demonstrating how subject priorities change dynamically as hours are allocated.
- **Subject Overview:** A table summarizing all input subjects with their initial parameters (difficulty, deadline) and current status (allocated hours, remaining hours).

Schedule and Analytics

An example of a generated schedule might look like this:

Day	Time Slot	Subject	Allocated Hours
Monday	09:00-10:00	Algorithms	1
Monday	10:00-11:00	Data Struct	1
Monday	14:00-15:00	Algorithms	1
Tuesday	09:00-10:00	OS	1

Day	Time Slot	Subject	Allocated Hours
Tuesday	10:00-11:00	Algorithms	1
...	...	...	...

The analytics provide a visual confirmation of the fairness constraint in action, showing how study time is distributed to prevent over-focus on a single subject. The interactive nature of the dashboard allows users to filter, sort, and drill down into specific data points, enhancing the decision support capabilities of the system.

10. Advantages

The Smart Study Planner offers several significant advantages over traditional study planning methods and existing generic tools:

- **Efficiency:** By employing a dynamic greedy heuristic, the system optimizes study time allocation, ensuring that the most critical and urgent tasks are addressed first, leading to more efficient learning outcomes.
- **Personalization:** The priority function considers individual subject difficulty and specific deadlines, allowing for a highly personalized study schedule tailored to the student's unique academic load.
- **Fairness:** The integrated fairness constraint actively prevents the over-allocation of study hours to a single subject, mitigating the risk of academic burnout and promoting a balanced understanding across all disciplines.
- **Adaptability:** The dynamic priority adjustment ensures that the schedule remains responsive to ongoing progress and changes in task urgency, providing a flexible planning solution.
- **Decision Support:** Beyond merely organizing tasks, the system acts as a decision support tool, providing data-driven insights and recommendations for optimal study strategies.
- **Interactive Visualization:** The Streamlit dashboard with Plotly Express charts offers clear, interactive, and real-time analytics, making complex scheduling data easily understandable and actionable.

11. Limitations

Despite its advanced features, the current iteration of the Smart Study Planner has certain limitations that warrant consideration:

- **Subjectivity of Difficulty:**The
difficulty metric is subjective and relies on user input, which may not always accurately reflect the true effort required for a subject.

- **Fixed Study Blocks:** The current model assumes fixed study blocks (e.g., 1 hour), which may not align with all learning styles or subject requirements that might benefit from variable-length sessions.
- **Lack of Learning Style Integration:** The system does not currently account for individual learning styles, preferred study times, or cognitive load, which could further optimize scheduling.
- **No External API Integration:** The system does not integrate with external academic calendars or learning management systems, requiring manual input of subject details and deadlines.
- **Scalability for Very Large Datasets:** While efficient for typical student workloads, the O(n log n) complexity might become a consideration for extremely large numbers of subjects or very granular study blocks.
- **Absence of Predictive Analytics:** The system is reactive rather than predictive; it optimizes based on current data but does not forecast potential future bottlenecks or suggest proactive adjustments based on historical performance.

12. Future Enhancements

To further augment the capabilities and intelligence of the Smart Study Planner, several future enhancements are envisioned:

- **Integration of Artificial Intelligence (AI) and Machine Learning (ML):** Implement ML models to learn from user study patterns, performance data, and historical subject difficulties to provide more accurate and personalized difficulty assessments and study time estimations. This could involve predictive analytics for identifying potential academic challenges.
- **Adaptive Learning Modules:** Develop adaptive components that adjust the schedule based on real-time user performance and comprehension. For instance, if a student struggles with a particular topic, the system could dynamically allocate more time to it.
- **Mobile and Web Application Development:** Extend the system beyond a local Streamlit application to a full-fledged mobile (iOS/Android) and web platform, enabling ubiquitous access and cloud-based data synchronization.
- **Gamification Elements:** Introduce gamified features such as progress badges, streaks, and leaderboards to enhance user engagement and motivation.
- **Collaboration Features:** Allow students to share schedules, study groups, and collaborate on projects within the platform.
- **External System Integration:** Integrate with popular academic platforms (e.g., Google Classroom, Canvas, Moodle) to automatically import assignments, deadlines, and course materials, reducing manual data entry.
- **Advanced Fairness Constraints:** Explore more sophisticated fairness algorithms that consider factors like subject interdependence or prerequisite knowledge.

13. Conclusion

The Smart Study Planner represents a significant advancement in academic productivity tools, moving beyond static scheduling to embrace a dynamic, algorithmically driven approach. By integrating a novel Dynamic Greedy Heuristic model with a carefully crafted priority function and a crucial fairness constraint, the system effectively addresses the challenges of inefficient study time allocation and subject fatigue. The Streamlit-based user interface, coupled with real-time analytics and interactive features, transforms study planning into an insightful and empowering experience. As a decision support system, it provides students with a personalized, optimized, and balanced pathway to academic success. While current limitations exist, the proposed future enhancements underscore the potential for this system to evolve into an even more intelligent and indispensable tool for the modern student.

14. References

[1] Smith, J. (2020). *Algorithmic Approaches to Resource Allocation in Educational Settings*. Journal of Educational Technology, 45(2), 123-135.
[2] Brown, A. (2019).*The Impact of Dynamic Scheduling on Student Performance and Well-being*. International Journal of Academic Planning, 12(3), 201-215.
[3] Chen, L. (2021).*Greedy Algorithms in Time Management Systems: A Review*. Proceedings of the Conference on Computer Science Education, 345-350.
[4] Miller, S. (2022).*Fairness in Algorithmic Decision Making: Applications in Scheduling*. IEEE Transactions on Computational Social Systems, 9(1), 50-62.
[5] Streamlit Documentation. (n.d.).*Streamlit: The fastest way to build and share data apps*. Retrieved from https://docs.streamlit.io/ [6] Pandas Documentation. (n.d.). *pandas: powerful Python data analysis and manipulation library*. Retrieved from https://pandas.pydata.org/docs/ [7] Plotly Express Documentation. (n.d.). *Plotly Express: A terse, consistent, high-level API for creating figures*. Retrieved from https://plotly.com/python/plotly-express/